

Università
Pegaso

Spiking Neural Networks

A primer

Salvatore Barone[1]

[1]Department of Comnputer Science and Technologies
Pegaso University

Lecturer

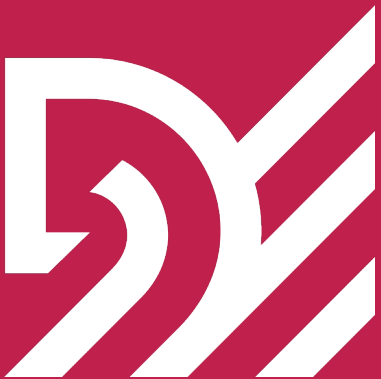


Salvatore Barone is Tenured Assistant Professor (RTT L. 240/2010, scientific area IINF-05/A - Information Processing Systems) at the **Department of Computer Science and Technologies**, **Pegaso University**. He attained his PhD in *Information Technology and Electrical Engineering* from *University of Naples Federico II*, in 2022.
His research activities include the of design of high-performance computing systems, embedded systems, and safety-critical systems. He participates to research activities of the *Center for Artificial Intelligence and Extended Reality* at Pegaso University.

1. A primer on Spiking Neural Networks
2. Information Encoding
3. Neuron Models
   - The Hodgkin-Huxley Neuron Model
   - Leaky Integrate and Fire
   - The Izhikevich Neuron Model
4. Learning
   - Spike Timing-Dependent Plasticity
   - Learning through backpropagation
5. Hand-on tutorial



1 A primer on Spiking Neural Networks

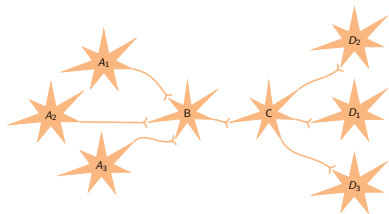


Neuromorphic computing

- Neuromorphic ("brain-like") engineering strives to **imitate** the computational principles of the **brain**
  - drive down the energy cost of artificial intelligence systems
- It builds on **three parts**:
  - Neuromorphic **sensors**: take inspiration from biological sensors, such as the retina or cochlear
  - Neuromorphic **algorithms** that learn to make sense of spikes are known as **Spiking Neural Networks**
  - Neuromorphic **hardware** is special hardware that takes advantages of Spiking Neural Networks.

Spiking Neural Networks

- Spiking Neural Networks consists of **neurons interconnected by synapses**
- Information is **encoded** and **transmitted** in the form of **spikes**
- Unlike the converional ANNs, SNNs **do not perform a single forward pass**
  - Each input is presented for a prespecified amount of time T
  - Each forward pass requires δT to be performed
  - Each inference requires $\frac{T}{\delta T}$ forward passes



2 Information Encoding



Information Encoding

- Spiking Neural Networks require to map information to a **discrete spike domain** before feature extraction
- Several encoding strategies, broadly divided in **two grups**: **Rate coding** and **Temporal Coding**
  - Advantages of Rate coding:
    - stronger gradient signal for learning via error backpropagation
    - more spikes reduce the burden of some neurons failing to spike
  - Advantages of temporal coding:
    - fewer spikes mean **less dynamic power dissipation** in tailored hardware
    - Sparsity reduces the power consumption of vector-matrix products

Daniel Auge, Julian Hille, Etienne Mueller, and Alois Knoll. "A Survey of Encoding Techniques for Signal Processing in Spiking Neural Networks".

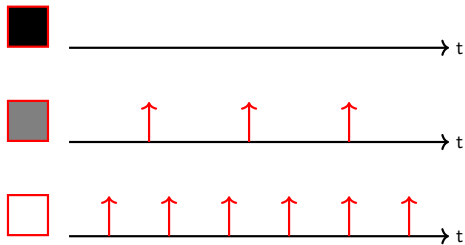
In: *Neural Processing Letters* 53.6 (2021). DOI: 10.1007/s11063-021-10562-2

Rate coding

- **Count rate**: the probability of spike occurrence at a given time instant t is modelled by a **Poisson distribution**

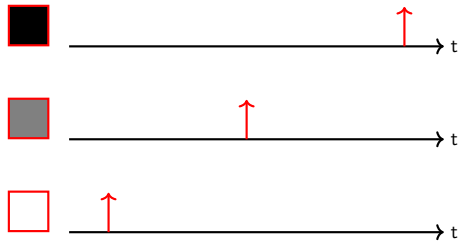
$$P_v(n) = \frac{v^n}{n!} e^{-v}, \quad v = \frac{N_{spike}}{T}$$

  - the mean event rate is proportional to the information to encode

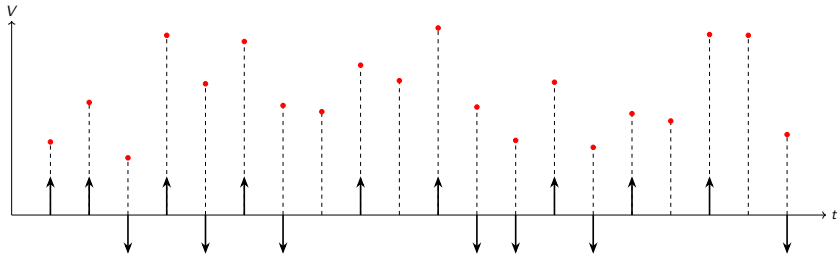


Temporal coding

- **Latency coding** or **Time to first spike**
  - Features closer to 1 will fire earlier and features closer to 0 will fire later
  - The bright pixels induce firing at the start of the run, and the dark pixels at the end.

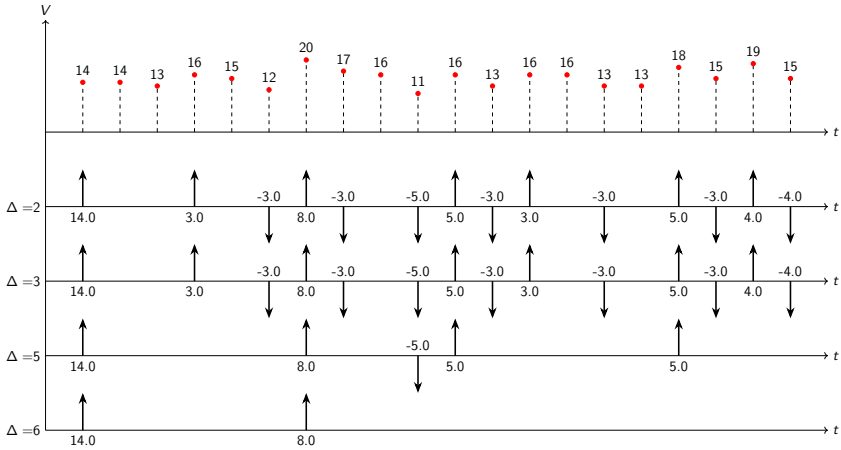


Simple Delta Modulation



- Whenever the change in the input exceeds this **threshold**, a spike is generated at those specific time steps.
- This can produce **positive** and **negative** spikes to reflect the direction of change.

n-Threshold Delta Modulation



- defines multiple predefined delta values corresponding to the degree of change in the input.
  - These deltas produce **n different spike trains**

3 Neuron Models

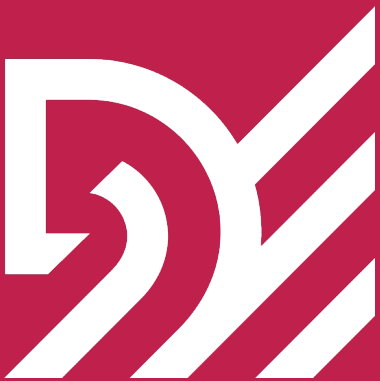


Computational model of spiking neurons

- The Hodgkin-Huxley neuron is the most accurate model
  - It models even ion channels and molecular interactions
- We need a model that sits in the sweet spot between biological plausibility and practicality.
- A large variety of neuron models are out there. Shared and common properties:
  - neurons have an **internal state** that **accumulates** the input stimuli
  - the neuron **fires** when the internal state crosses a **threshold** value
  - firing event is followed by a **refractory time-period**

Wulfram Gerstner and Werner M. Kistler. *Spiking Neuron Models: Single Neurons, Populations, Plasticity*. 2002

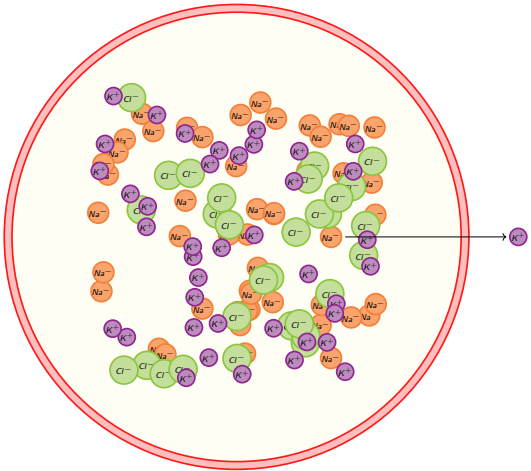
3 Neuron Models
- The Hodgkin-Huxley Neuron Model



The Hodgkin-Huxley Neuron Model

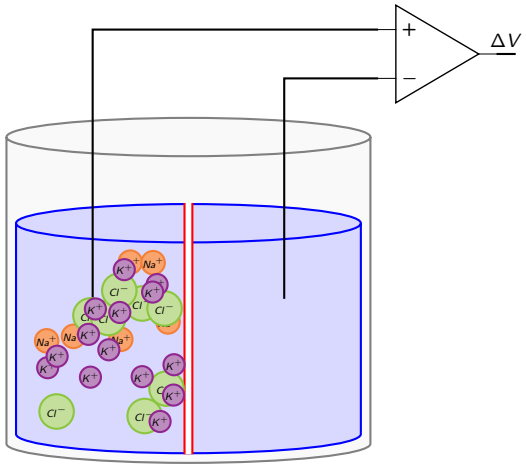
- The Hodgkin-Huxley model uses electrical component to describe the behavior of the neuron.
- It is biologically derived, but it is a model:
  - It can simulate different neurons with high accuracy, but it is not 100% accurate
  - There are different neurons (different genes, different ion channels, different morphology)
  - Each gene makes a particular neurons for a specific purpose (visual cortex, motor cortex, etc.)
  - Each neuron responds to input stimuli in a different way

Charges drift in saline solutions



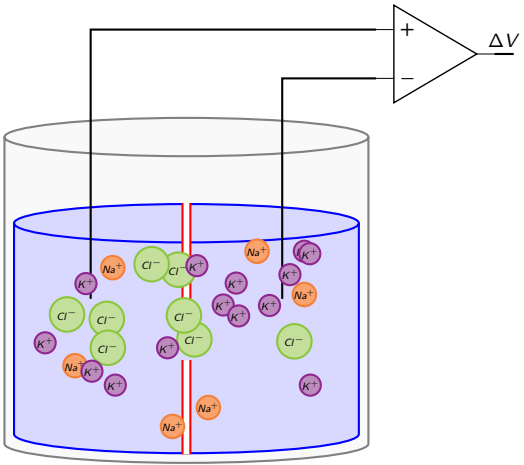
- Ions can diffuse from the inside of the cell to the outside of the cell
  - Through **ion-selective channels**
- Each particle diffuses independently
- Drift current in saline solutions obey the Ohm's law!

Ion-specific channels



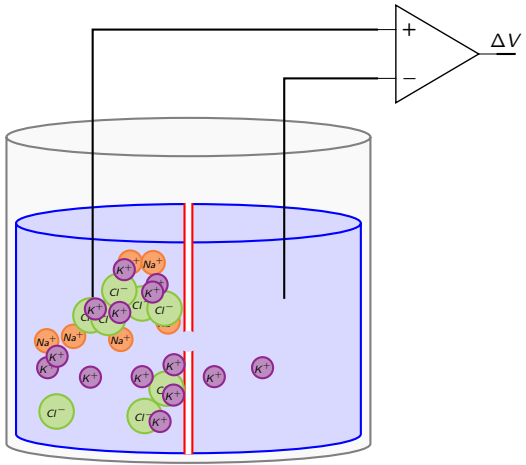
- Consider a beaker with a saline solution
- The beaker is separated in two regiosn using a **membrane**
- K^+ , Na^- and Cl^- ions are only on the left region of the beaker

Ion-specific channels



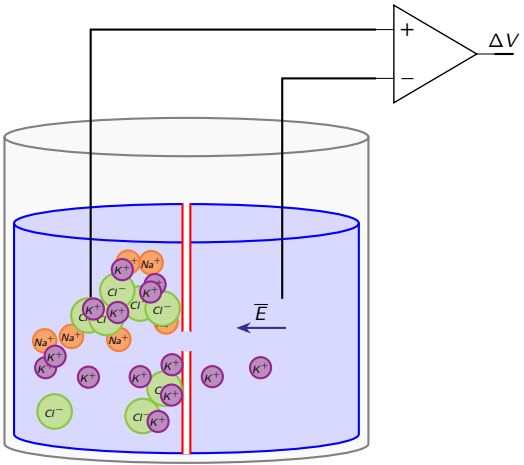
- If we make a **non ion-specific hole** in the membrane, ions are going to diffuse from high concentration regions (left) to low concentration regions (right)
- **Eventually**, the concentration of ions in the two regions becomes equal, reaching the **equilibrium**

Ion-specific channels



- If we make an **ion-specific hole** in the membrane
  - e.g. K^+ specific hole, some ions are going to diffuse
- Only some K^+ ions are going to diffuse
- The concentration never reaches the equilibrium

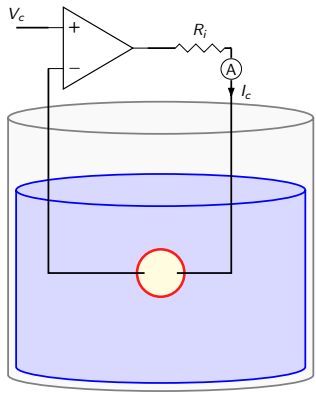
Nernst Potential



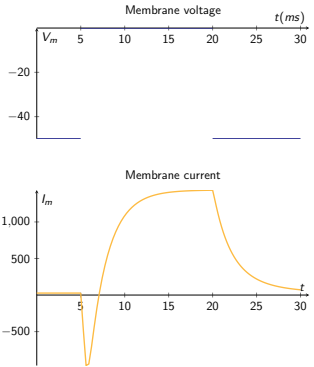
- A voltage difference between the two sides develops when making the hole in the membrane
- There will be a flow of ions drifting due to the electric field E_{K^+} from the right to the left side
- This balances the flow of K^+ ions diffusing to the right side due to the concentration gradient $E_{K^+} \approx -75$ mv

The Voltage Clamp Experiment

- The best way to study the voltage and time dependance is to suddenly set the membrane potential to a control voltage V_c and measure the current I_c required to hold that voltage
  - We step V_c , we measure I_c , we plot the V_c - I_c relationship and we extract the conductance of the membrane
- **Hyperpolarization** (step from -65mV to -120mV) and nothing happens...
- **Depolarization** (step from -65mV to 0mV) and **something happens**!

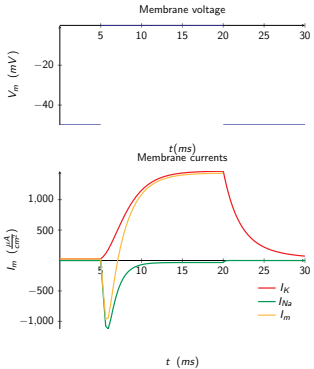


Potassium and Sodium channels



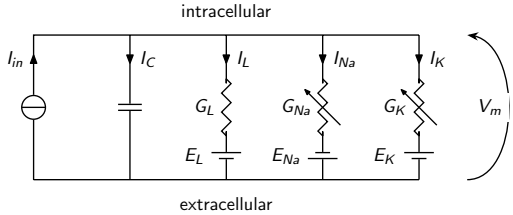
- They observed a negative current (positive ions moving inside the cell)
- The current eventually reverses and approaches a constant value
- The hypotesis from Hodgkin and Huxley was that the observed current was the sum of **two contributions**:
  - One from potassium channels and one from sodium channels

Potassium and Sodium channels



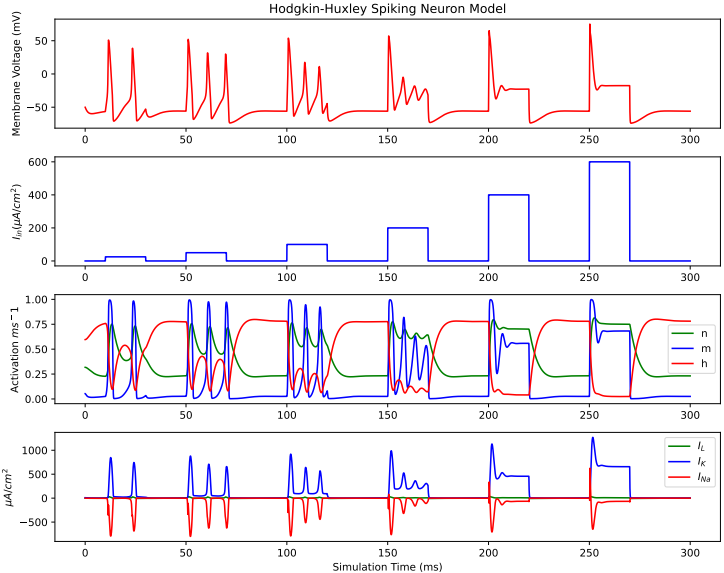
- To highlight the contributions, they **replaced sodium cloride** ($NaCl$) whith **choline cloride** ($C_5H_{14}NOCl$) in the extracellular solution
- They observed the very same behavior except that the initial negative pulse went away
- **The negative current is due to sodium channels!**

Equivalent Circuit

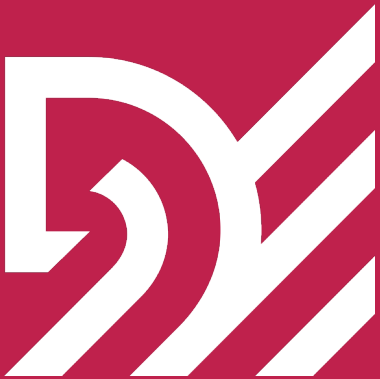


- The Hodgkin-Huxley model uses electrical component to describe the behavior of the neuron.
- A lipid bilayer that insulates the conductive saline solution within the neuron from the extracellular (conductive) medium
- There are specific **channels in the membrane** that can be **triggered to open** by injecting **current**
  - Two conductive solutions separated by an insulator act as a **capacitor**
  - Charge movement can be modelled using a **resistor**
- Time- and voltage-dependent variable conductances generate action potential!

Simulation


Hodgkin-Huxley Spiking Neuron Model
Membrane Voltage (mV)
$I_{in}(\mu A/cm^2)$
Activation ms^{-1}
n
m
h
$\mu A/cm^2$
I_L
I_K
I_{Na}
Simulation Time (ms)

3 Neuron Models
- Leaky Integrate and Fire

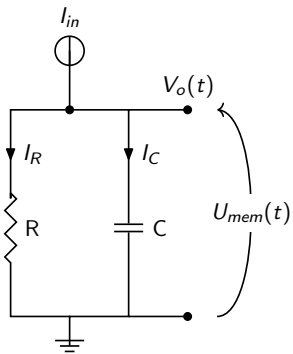


Leaky Integrate and Fire

- Leaky Integrate and Fire can be derived from the Hodgkin Huxley model by removing the adjustable conductances from the model
  - Two model: 1st-order and 2nd-order Leaky Integrate and Fire

1st-order Leaky Integrate and Fire

- Assume that an input voltage spike leads to an instantaneous jump in synaptic current, which then contributes to the membrane potential

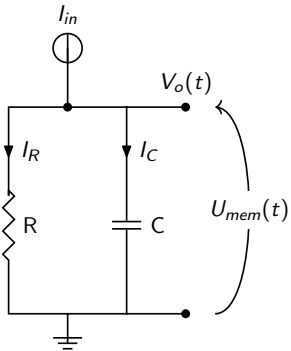


$$V_m[t+1] = \beta V_m[t] + WX[t+1] - S[t]V_{thr}$$

$$S[t] = \begin{cases} 1 & \text{if } V_m[t] > V_{thr} \\ 0 & \text{if } V_m[t] \leq V_{thr} \end{cases}$$

2nd-order Leaky Integrate and Fire

- Accounts for the **temporal dynamics of input current**



$$I_{syn}[t+1] = \alpha I_{syn}[t] + WX[t+1]$$

$$V_m[t+1] = \beta V_m[t] + I_{syn}[t+1] - S[t]V_{thr}$$

$$S[t] = \begin{cases} 1 & \text{if } V_m[t] > V_{thr} \\ 0 & \text{if } V_m[t] \leq V_{thr} \end{cases}$$

3 Neuron Models
- The Izhikevich Neuron Model



The Izhikevich Neuron Model

- Using biophysically accurate Hodgkin-Huxley neuron model is computationally prohibitive, since we can simulate only a handful of neurons in real time.
- Using Leaky Integrate and Fire model is computationally effective, but the model is unrealistically simple and incapable of producing rich spiking and bursting dynamics exhibited by cortical neurons.
- The Izhikevich model is biologically plausible as the Hodgkin-Huxley neuron model, yet as computationally efficient as the Leaky Integrate and Fire
  - The model reproduces spiking and bursting behavior of known types of cortical neurons

The Izhikevich Neuron Model

- Two-dimensional system of ordinary differential equations

$$\frac{\partial}{\partial t} v = 0.04 \cdot v^2 + 5 \cdot v + 140 - u + I$$
$$\frac{\partial}{\partial t} u = a \cdot (b \cdot v - u)$$

- After-spike reset mechanism

$$\text{if } v \geq 30mV \Rightarrow \begin{cases} v = c \\ u = u + d \end{cases}$$

Discretization

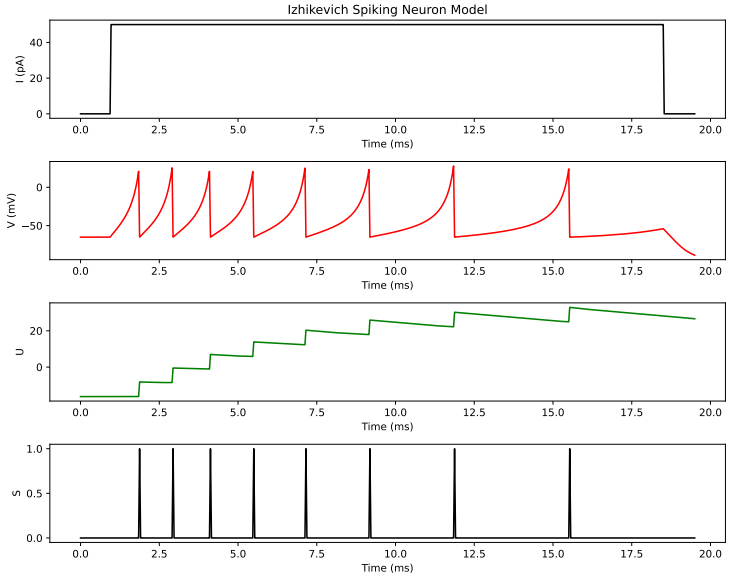
- Two-dimensional system of ordinary equations

$$v(t+1) = \left(0.04 \cdot v(t)^2 + 5 \cdot v(t) + 140 - u(t) + I(t)\right) \cdot T$$
$$u(t+1) = a \cdot (b \cdot v(t) - u(t)) \cdot T$$

- After-spike reset mechanism

$$\text{if } v \geq 30mV \Rightarrow \begin{cases} v(t) = c \\ u(t+1) = u(t+1) + d \end{cases}$$

Simulation


Izhikevich Spiking Neuron Model
I (pA)
V (mV)
U
S
Time (ms)
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0

4 Learning



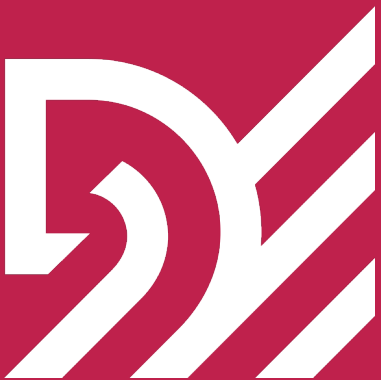
Learning

- The exact mechanisms by which biological neural networks learn are an open research question
- Consider a pre-synaptic neuron j and a post-synaptic neuron i
  - The weight w_{ij} of a connection from neuron j to i is considered as a parameter
  - Parameters are adjusted so as to optimize the performance of a network for a given task.
  - The process of parameter adaptation is called **learning**
  - The procedure for adjusting the weights is referred to as a **learning rule**.

Learning Rules

- Broadly Speaking, Spiking Neural Networks can be trained using threee different approaches
  - **Converting ANNs to SNNs**: non spiking neural networks are converted to spiking neural networks
    - The conventional activations are interpreted as firing rate or spike time-instant
  - Backpropagating the error
    - Backprop Through Time on Backprop using Spikes
  - Using **biologically plausible** learning rules
    - The weight update is a function of the spatial and temporal behavior of single neurons

4 Learning
- Spike Timing-Dependent Plasticity



Spike Timing-Dependent Plasticity

- Hebbian learning rule:
  - "*When an axon of Cell A is near enough to excite a Cell B and repeatedly or persistently takes part in firing it, some growth process or metabolic change takes place in one or both cells such that A's efficiency, as one of the cells firing B, is increased*" – The Organization of Behaviour: A Neuropsychological Theory, D. Hebb

Natalia Caporale and Yang Dan. "Spike Timing–Dependent Plasticity: A Hebbian Learning Rule". In: *Annual Review of Neuroscience* 31.1 (2008).

DOI: 10.1146/annurev.neuro.31.060407.125639

Synaptic plasticity

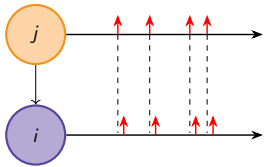
- The resulting change in the synaptic efficacy Δw_{ji} after several repetitions of the experiment turns out to be a function of the spike time differences $t(f)_j - t(f)_i$
  - Synaptic changes Δw_{ij} occur only if presynaptic firing at $t(f)_j$ and postsynaptic activity at $t(f)_i$ occur **sufficiently close** to each other.
  - The direction of the change depends critically, i.e., **on a millisecond time-scale**, on the relative timing of pre- and post-synaptic spikes.
- **Hebbian plasticity**:
  - the synapse is strengthened if the presynaptic spike occurs shortly before the postsynaptic neuron fires
  - The synapse is weakened if the sequence of spikes is reversed
- **anti-Hebbian plasticity**:
  - the synapse is weakened if the presynaptic input arrives shortly before the postsynaptic spike

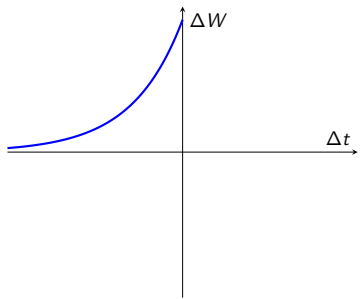
Mathematical formulation

- Three relevant aspects of the Hebbian learning:
  - **Locality** means that the change of the synaptic efficacy can only depend on local variables
    - i.e., on information that is available at the site of the synapse
    - pre- and post- synaptic firing rate, and the actual value of the synaptic efficacy, but not on the activity of other neurons.
  - **Cooperativity** implies that preand postsynaptic neurons have to be active simultaneously for a synaptic weight change to occur.
  - **Competition** synaptic weights can only grow at the expense of others so that if a certain subgroup of synapses is strengthened, other synapses to the same postsynaptic neuron have to be weakened.

Mathematical formulation

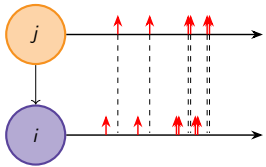
- t_{pre} the time instant of the last pre-synaptic spike
- t_{post} the time instant of the last post-synaptic spike
- $\Delta t = t_{pre} - t_{post}$ the difference between the last two presynaptic and postsynaptic spike events
  - If $t_{post} > t_{pre} \Leftrightarrow \Delta t < 0$, the **presynaptic** spike **happened before** the **postsynaptic** one, then
  - There might be correlation
  - We want the synapse to be potentiated

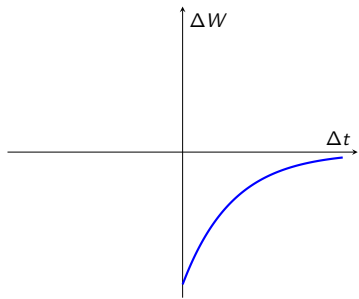




Mathematical formulation

- t_{pre} the time instant of the last pre-synaptic spike
- t_{post} the time instant of the last post-synaptic spike
- $\Delta t = t_{pre} - t_{post}$ the difference between the last two presynaptic and postsynaptic spike events
  - Conversely, If $t_{post} \leq t_{pre} \Leftrightarrow \Delta t \geq 0$, the post synaptic spike happened before the presynaptic one
  - there is no correlation between the events
  - we want the synapse to be depressed



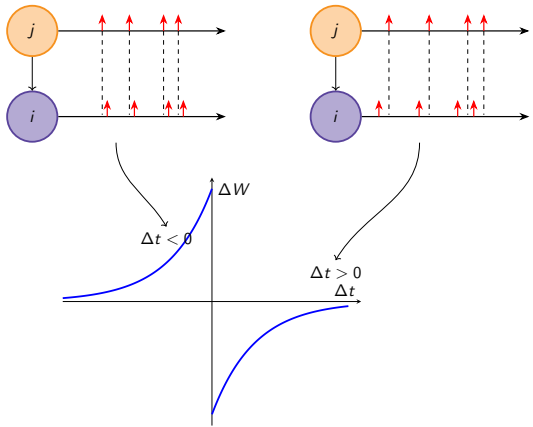


Exponential Learning window

- **Exponential learning window**:

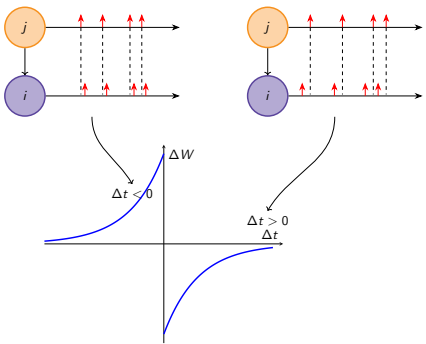
$$\Delta W = \begin{cases} A_+ \cdot e^{\frac{\Delta t}{\tau_+}} & \text{if } \Delta t < 0 \\ -A_- \cdot e^{\frac{-\Delta t}{\tau_-}} & \text{if } \Delta t \geq 0 \end{cases}$$

  - $A_+ > 0$, $A_- > 0$ are the **learning rates**, and
  - $\tau_+ > 0$ and $\tau_- > 0$ are the **decay time-constants**



Soft bounds for the synaptic weights

- Electrophysiological experiments, however, show that the **response amplitude is not fixed** but can change over time.
- Synaptic efficacy must stay within certain bounds
  - The synapse is thus no longer strengthened (or weakened) if the weight reaches its upper (lower) bound
    - $A_+ \propto (1 - w_{ij}) \cdot a_+$
    - $A_- \propto w_{ij} \cdot a_-$



4 Learning
- Learning through backpropagation



Learning through backpropagation

- The most impressive results reported in the literature usually resort to **Supervised learning** techniques **using approximate backpropagation**
- **Backpropagation is not biologically feasible**
  - in each iteration, a f**irst-order Taylor approximation** of the **loss function** to be optimized is applied.
  - backpropagating the error would require **bidirectional synapses**
  - learning in the brain occurs continuously, in a single step, while **backpropagation is a 2-step algorithm**
  - etc. etc. etc.
- Applying backpropagation to SNNs requires **computing** the **membrane voltage** for each neuron **at each time step**
  - The **backpropagation through time** algorithm

$$\frac{\partial \mathcal{L}}{\partial W} = \sum_t \frac{\partial \mathcal{L}[t]}{\partial W}$$

The Dead Neuron Problem

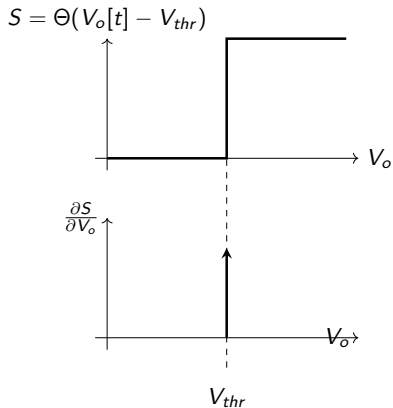
- Using the backpropagation algorithm to train the network implies using the **gradient** of the loss with respect to the weights

- the weights are updated to minimize the loss
- The backpropagation algorithm achieves this using the chain rule

$$\frac{\partial \mathcal{L}}{\partial W} = \frac{\partial \mathcal{L}}{\partial S}\frac{\partial S}{\partial U}\frac{\partial V_o}{\partial I}\frac{\partial I}{\partial W}$$

- The $S[t]$ function can be represented using the **Heaviside step function**

$$S[t] = \Theta(V_o[t] - V_{thr})$$

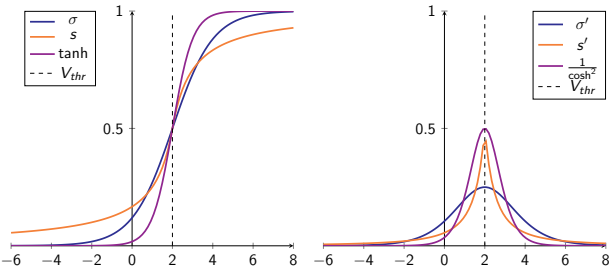


The surrogate gradient approach

- The most common approach to overcome the The dead neuron problem is
  - keep the Heaviside function during the forward pass
  - replace the derivative term with a function that **does not nullify the learning process**

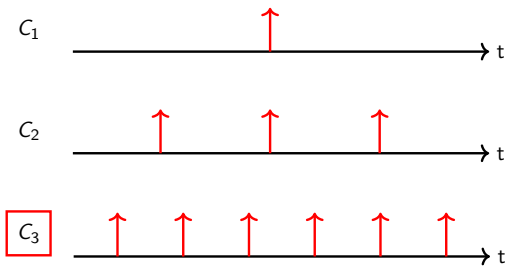
$$\frac{\partial S}{\partial V_o} \leftarrow \frac{\partial \tilde{S}}{\partial U} = \frac{1}{(1 + k\,|U_{od}|)^2}$$

  - $U_{OD} = V_o - V_{thr}$ is the **overdrive** of the membrane potential



Learning through backpropagation: loss functions on rate-count

- Suppose the correct class is class i , we want to apply the target to the i-th class, i.e.,
  - The membrane potential of the correct class has to be increased above the threshold, i.e, $V_{o_i} > V_{thr}$
  - The membrane potential of incorrect classes must be kept below the threshold, i.e., $V_{o_j} \leq V_{thr} \forall j \neq i$
- Two alternatives: **spiking rate** or **membrane potential** to define the loss function



Cross entropy on **rate count**

- Consider the **spike count** from each output neuron
  - can be obtained by summing $\boldsymbol{S}[t]$ over T time steps

$$\boldsymbol{c} = \sum_{j=0}^{T} \boldsymbol{S}[t]$$

- The $c_i \in \boldsymbol{c}$ are treated as *logits* to compute the *softmax*, i.e.,

$$p_i = \frac{e^{c_i}}{\sum_{j=0}^{N_C-1} e^{c_j}}$$

- Given the the target class $y_i = \{0,1\}^{N_C}$,

$$\mathcal{L}_{CE} = -\sum_{j=0}^{N_C-1} y_j \cdot log\ p_j$$

Cross entropy on **membrane potential**

- Logits are obtained by taking the maximum value of the membrane potential over time, i.e.,

$$\boldsymbol{v} := \{v_i := \max_{t=0}^{T}\{V_{o_i}[t]\},\ i \in [0, N_C - 1]\}$$

  - The index of $\boldsymbol{v}$ with the maximum count corresponds to the **predicted class**

$$\hat{y} = \arg\max_i\{\boldsymbol{v}\}$$

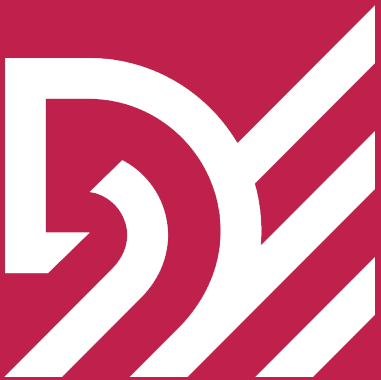
- Then, taking the softmax on logits

$$p_i = \frac{e^{v_i}}{\sum_{j=0}^{N_C-1} e^{v_j}}$$

- Given the the target class $y_i = \{0, 1\}^{N_C}$

$$\mathcal{L}_{CE} = -\sum_{j=0}^{N_C-1} y_j \cdot log\ p_j$$

5 Hand-on tutorial



Loading The MNIST handwritten digits



```
def get_MNIST():
    data_path='/tmp/data/mnist'
    transform = transforms.Compose([
            transforms.Resize((28, 28)),
            transforms.Grayscale(),
            transforms.ToTensor(),
            transforms.Normalize((0,), (1,))])
    mnist_train = datasets.MNIST(data_path, train=True, download=True, transform=transform)
    mnist_test = datasets.MNIST(data_path, train=False, download=True, transform=transform)
    return mnist_train, mnist_test

## Setting up the stati MNIST
mnist_train, mnist_test = get_MNIST()

# Training Parameters
batch_size=128

# Create DataLoaders
train_loader = DataLoader(mnist_train, batch_size=batch_size, shuffle=True, drop_last=True)

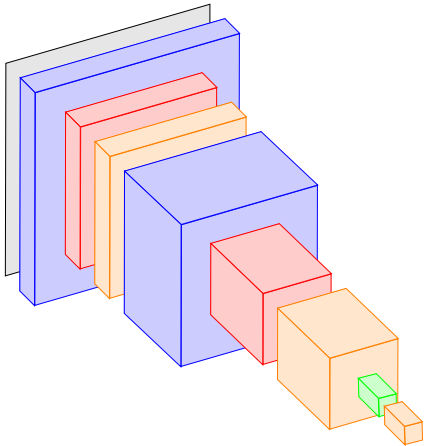
test_loader = DataLoader(mnist_test, batch_size=batch_size, shuffle=True, drop_last=True)
```

Convolutional Spiking Neural Networks

```
spike_grad = surrogate.fast_sigmoid(slope=25)

net = nn.Sequential(
    nn.Conv2d(1, 12, 5),
    nn.MaxPool2d(2),
    snn.Leaky(  beta=beta,
                spike_grad=spike_grad,
                init_hidden=True),
    nn.Conv2d(12, 64, 5),
    nn.MaxPool2d(2),
    snn.Leaky(  beta=beta,
                spike_grad=spike_grad,
                init_hidden=True),
    nn.Flatten(),
    nn.Linear(64*4*4, 10),
    snn.Leaky(  beta=beta,
                spike_grad=spike_grad,
                init_hidden=True,
                output=True))

net.to(device)
```



12C5-MP2-64C5-MP2-1024FC10

The forward pass

- SNNs process inputs over a time winwod
  - The forward pass requires a separate loop over time

```
def forward_pass(net, num_steps, data):
  mem_rec = []
  spk_rec = []
  utils.reset(net)  # resets hidden states for all LIF neurons in net

  for step in range(num_steps):
      spk_out, mem_out = net(data)
      spk_rec.append(spk_out)
      mem_rec.append(mem_out)

  return torch.stack(spk_rec), torch.stack(mem_rec)
```

- Wrapping the forward pass allows easy access to it during the training loop

```
spk_rec, mem_rec = forward_pass(net, num_steps, data)
```

The loss function and the optimizer

- When presented with some training data, our untrained network is likely not to give the correct answer.
  - **Loss function** measures the degree of dissimilarity of obtained result to the target value, and it is the loss function that we want to minimize during training.
  - To calculate the loss we make a prediction using the inputs of our given data sample and compare it against the true data label value.
  - We use **cross-entropy on spike rate here**

```
loss_fn = SF.ce_rate_loss()
...
loss_val = loss_fn(spk_rec, targets)
```

- Optimization is the process of adjusting model parameters to reduce model error in each training step.
  - All optimization logic is encapsulated in the **optimizer** object.

```
optimizer = torch.optim.Adam(net.parameters(), lr=1e-2, betas=(0.9, 0.999))
```

The training loop

```
for epoch in range(num_epochs):
    for data, targets in iter(train_loader):
        data = data.to(device)
        targets = targets.to(device)
        net.train()
        spk_rec, _ = forward_pass(net, num_steps, data)
        loss_val = loss_fn(spk_rec, targets)
        optimizer.zero_grad()
        loss_val.backward()
        optimizer.step()
        loss_hist.append(loss_val.item())
        if counter % 50 == 0:
          with torch.no_grad():
              net.eval()
              test_acc = batch_accuracy(test_loader, net, num_steps)
              print(f"Iteration {counter}, Test Acc: {test_acc * 100:.2f}%\n")
              test_acc_hist.append(test_acc.item())
        counter += 1
```

The training loop

```
Device: cuda
Network model:
Sequential(
  (0): Conv2d(1, 12, kernel_size=(5, 5), stride=(1, 1))
  (1): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (2): Leaky()
  (3): Conv2d(12, 64, kernel_size=(5, 5), stride=(1, 1))
  (4): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (5): Leaky()
  (6): Flatten(start_dim=1, end_dim=-1)
  (7): Linear(in_features=1024, out_features=10, bias=True)
  (8): Leaky()
)
Performing the training loop with 1 epochs.
Batch size is 128.
Learning rate is set to 0.01
Iteration 0, Test Acc: 9.81%

Iteration 50, Test Acc: 92.99%

Iteration 100, Test Acc: 95.39%
```

A note on hyperparameters

- Hyperparameters are adjustable parameters that let you control the model optimization process.
  - **Number of Epochs**: the number of times to iterate over the dataset.
  - **Batch Size**: the number of data samples propagated through the network before the parameters are updated
  - **Learning rate**: how much to update models parameters at each batch/epoch

Testing the accuracy

```
def compute_accuracy(dataloader, net, num_steps):
  with torch.no_grad():
    total = 0
    acc = 0
    net.eval()
    train_loader = iter(dataloader)
    for data, targets in dataloader:
      data = data.to(device)
      targets = targets.to(device)
      spk_rec, _ = forward_pass(net, num_steps, data)
      acc += SF.accuracy_rate(spk_rec, targets) * spk_rec.size(1)
      total += spk_rec.size(1)
  return acc/total
```

Accuracy on the test set

```
Device: cuda
Network model:
Sequential(
  (0): Conv2d(1, 12, kernel_size=(5, 5), stride=(1, 1))
  (1): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (2): Leaky()
  (3): Conv2d(12, 64, kernel_size=(5, 5), stride=(1, 1))
  (4): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (5): Leaky()
  (6): Flatten(start_dim=1, end_dim=-1)
  (7): Linear(in_features=1024, out_features=10, bias=True)
  (8): Leaky()
)
Performing the training loop with 1 epochs.
Batch size is 128.
Learning rate is set to 0.01
Iteration 0, Test Acc: 9.81%

Iteration 50, Test Acc: 92.99%

Iteration 100, Test Acc: 95.39%

Iteration 150, Test Acc: 95.87%

Iteration 200, Test Acc: 96.93%

Iteration 250, Test Acc: 96.69%

Iteration 300, Test Acc: 97.13%

Iteration 350, Test Acc: 97.28%

Iteration 400, Test Acc: 96.74%

Iteration 450, Test Acc: 97.68%

Accuracy on test set images: 97.09
(venv)
```

\Huge Thank you!

Spiking Neural Networks

Salvatore Barone[1]

[1]Department of Comnputer Science and Technologies
Pegaso University

